```
  vehicle_id WITH =,
  tstzrange(starts_at, ends_at) WITH &&
)
WHERE (not canceled);
```

Let's break down what's happening in this Exclusion constraint.

The constraint is named "non_overlapping_vehicle_registration". The btree_gist extension is enabled because a GiST index is needed.

Identical vehicle_id values are selected using the WITH = clause and operator.

For the same vehicle_id, the starts_at and ends_at column values are compared using the WITH && clause and operator.

The && operator is a special one that identifies overlaps. Overlaps are elements that are in common.

The last piece part of the SQL statement is a WHERE clause with the WHERE (not canceled) condition. This clause is important because it excludes canceled Vehicle Reservations from this constraint definition. You'd want to allow overlapping Vehicle Reservation for reservations that have been canceled so that the business can collect revenue during that time.

And there you have it!

The Exclusion constraint is a more complicated but expressive constraint type that helps you prevent bad data from entering your system.

In the next section, you'll switch gears away from database constraints and move into the Active Record application layer.

Active Record provides a powerful and flexible set of built-in Validation types. Besides the built-in types, the system is extensible allowing you to write Custom Validators. How might you go about that?

Creating Active Record Custom Validators

Besides the built-in Active Record validations, Active Record supports Custom Validators.

Custom Validators[10] are created using object oriented design concepts.

A Ruby class that extends the ActiveModel::Validator class and implements a validate() instance method is a Custom Validator. The .validate() method signature accepts a single record argument.

10. https://guides.rubyonrails.org/active_record_validations.html#performing-custom-validations